

Каталог заметок

Заметки по embedded Linux, ядру, драйверам и микроконтроллерам.

Table of Contents

Поиск	1
Добавление новой статьи	1
Статьи	1
device tree	1
inzenerka tech	2
kernel	2
licheerv nano	2
milandr	3

Поиск

- В VS Code: Ctrl+Shift+F по тегу (например, `:keywords: gpio`)
- В браузере: открыть `file:index.html`[интерактивный каталог] — фильтрация по тегам и поиск

Добавление новой статьи

1. Создай `.adoc` файл в нужной папке, обязательно укажи `:keywords:` с тегами в заголовке
2. Названия файлов `.adoc` должны быть без пробелов
3. (опционально) Сгенерируй PDF: Ctrl+Shift+B в VS Code (asciidoctor-pdf)
4. Обнови каталог: `powershell -File scripts/generate-index.ps1` — автоматически обновятся README.adoc и index.html

Статьи

device tree

- [Модификация DTB на SD-карте](#) — Есть у нас образ и нужно нам поменять конфигурацию железа. **Теги:** `device-tree`, `dtb`, `sd-card`, `boot`, `pinmux`, `dtc`, `licheerv_nano`
- [Создание Device Tree Overlay](#) — Информация может быть и скорее всего не точна и даже может обманывать, получена из интернета и ИИ. **Теги:** `device-tree`, `overlay`, `dto`, `configs`, `pinmux`, `dtc`, `sg2002`

inzenerka tech

- [Разработка драйверов в Linux](#) — Материал написан на основе курса "Разработка модулей ядра Linux (Linux Kernel developer)" от <https://inzhennerka.tech/> и собранных данных в интернете. **Теги:** `kernel`, `driver`, `module`, `syscall`, `char-device`, `procfs`, `sysfs`, `printk`
- [Разработка драйверов в Linux, 1](#) — Материал написан на основе курса "Разработка модулей ядра Linux (Linux Kernel developer)" от <https://inzhennerka.tech/> и собранных данных в интернете. **Теги:** `kernel`, `driver`, `module`, `makefile`, `debugging`, `ftrace`, `kprobe`, `kgdb`, `kasan`, `oops`
- [Разработка драйверов в Linux, 2](#) — Материал написан на основе курса "Разработка модулей ядра Linux (Linux Kernel developer)" от <https://inzhennerka.tech/> и собранных данных в интернете. **Теги:** `kernel`, `driver`, `module`, `kernel_symvol`, `параметры_модуля`, `Module.symvers`
- [Теория по linux](#) — `nm` - позволяет посмотреть функции и тп. в бинарном файле, стоит смотреть флаги T-что то типа внутренних функций, U-сторонние функции **Теги:** `linux`, `kernel`, `gcc`, `makefile`, `library`, `static`, `shared`, `ipc`, `pipe`, `fifo`, `message-queue`, `shared-memory`, `filesystem`, `syscall`

kernel

- [Module.symvers](#) — Module.symvers (Symbol Versions)* — это глобальный реестр (словарь) всех функций и переменных ядра, которые разрешено использовать внешним модулям. **Теги:** `driver`, `module`, `kernel_symvol`, `Module.symvers`
- [Как задействовать Module.symvers из ядра](#) — На сколько я понимаю, ядро мы дополняем модулями, в каждом модуле есть некие функции, некоторые из этих функций мы можем вызывать снаружи. **Теги:** `driver`, `module`, `kernel_symvol`, `Module.symvers`
- [Руководство по навигации и поиску в исходниках ядра Linux](#) — Разработка модулей ядра Linux требует умения эффективно работать с исходным кодом и документацией. Ядро содержит миллионы строк кода, тысячи функций и обширную документацию. Умение быстро находить нуж... **Теги:** `kernel`, `documentation`, `elixir`, `bootlin`, `ctags`, `cscope`, `navigation`
- [Символы ядра \(Kernel Symbols\)](#) — Когда разработчики Linux пишут код ядра, они пишут его на языке C. Там есть тысячи функций: `sys_read`, `do_fork`, `tcp_sendmsg` и переменные. Когда этот код компилируется, он превращается в машинные нули и... **Теги:** `kernel`, `symbols`, `kallsyms`, `module`, `cfi`

licheerv nano

- [Полезные команды](#) — # 1. Проверяем права доступа к `/dev/mem` **Теги:** `licheerv_nano`, `commands`, `devmem`, `memory`, `debug`, `tail`
- [Работа с GPIO в Linux](#) — Данные заметки созданы как дополнительная напоминка и могут нести ошибки и неверное понимание автором как теории, так и практических полученных данных, а потому если попытаетесь повторить необходимо ... **Теги:** `gpio`, `licheerv_nano`, `pinmux`, `sysfs`, `libgpiod`, `gpiochip`, `output`, `input`

- [Работа с GPIO в Linux в режиме ШИМ](#) — И так сначала мы занимались ногодрыгом (gpio_output.pdf), достучались до пинов. Далее создали файл инициализирующий его режим gpio при загрузке системы. **Теги:** `pwm`, `licheerv_nano`, `pinmux`, `sysfs`, `sg2002`
- [Создание скрипта инициализации для BusyBox](#) — Если ядро не позволяет менять Device Tree на лету, мы говорим системе: *"Сделай это через 2 секунды после загрузки, пока пользователь еще даже не успел залогиниться"*. **Теги:** `busybox`, `init`, `sysvinit`, `startup`, `daemon`, `licheerv_nano`

milandr

- [Сборка и прошивка Миландр 1986VE9x через Docker](#) — Проект предназначен для сборки и прошивки микроконтроллера **Миландр 1986VE9x** (Cortex-M3) с использованием Docker-контейнера. Это обеспечивает: **Теги:** `milandr`, `1986ve9x`, `cortex-m3`, `docker`, `openocd`, `jlink`, `firmware`

Сгенерировано автоматически. Всего статей: 15.